



Московский государственный университет имени М.В. Ломоносова

Факультет вычислительной математики и кибернетики

Кафедра суперкомпьютеров и квантовой информатики

Отчет по заданию практикума методы Монте-Карло

Группа: 523

Повжик Юрий Максимович
Почта: s02210514@gse.cs.msu.ru

Москва, 2025

Содержание

1	Программный код.....	3
2	Графики	5
2.1	Фиксированное число процессов.....	5
2.2	Фиксированное число частиц	7
2.3	Число частиц = 10^3 * число процессов	9

1 Программный код

```
#include <mpi.h>
#include <stdlib.h>
#include <time.h>
#include <iostream>
#include <fstream>

using namespace std;

bool try_luck(double p) {
    return ((rand() / double(RAND_MAX)) < p);
}

int my_walk(int a, int b, int x, double p, long& stepCounter) {
    while (x > a && x < b) {
        if (try_luck(p)) {
            x += 1;
        } else {
            x -= 1;
        }
        stepCounter++;
    }
    return x;
}

int main(int argc, char** argv) {
    int size;
    int rank;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (argc != 6) {
        if (rank == 0) {
            cout << "Initial data is needed" << endl;
        }
        MPI_Finalize();
        return 0;
    }

    int a = atoi(argv[1]);
    int b = atoi(argv[2]);
    int x = atoi(argv[3]);
    double p = atof(argv[4]);
    int N = atoi(argv[5]);

    if (p < 0 || p > 1 || N <= 0 || a > b) {
        if (rank == 0) {
```

```

        cout << "Bad start data" << endl;
    }
    MPI_Finalize();
    return 0;
}

srand(time(NULL)*(rank+1));

long stepCounter = 0;
int exitRightCounter = 0;
int repNum = N/size;
if ((N % size) > rank) {
    repNum++;
}

double startTime, fullTime;
startTime = MPI_Wtime();
for(int i = 0; i < repNum; i++) {
    int pos = my_walk(a, b, x, p, stepCounter);
    if (pos == b) {
        exitRightCounter++;
    }
}
fullTime = MPI_Wtime() - startTime;

long stepCounterGl;
int exitRightCounterGl;
double fullTimeGl;

MPI_Reduce(&stepCounter, &stepCounterGl, 1, MPI_LONG, MPI_SUM, 0,
MPI_COMM_WORLD);
MPI_Reduce(&exitRightCounter, &exitRightCounterGl, 1, MPI_INT, MPI_SUM, 0,
MPI_COMM_WORLD);
MPI_Reduce(&fullTime, &fullTimeGl, 1, MPI_DOUBLE, MPI_MAX, 0,
MPI_COMM_WORLD);

if (rank == 0) {
    ofstream outTxt("output.txt");
    if(outTxt.is_open()) {
        double exitRightCounterGlDouble = exitRightCounterGl;
        outTxt << "Вероятность достижения состояния b: " <<
exitRightCounterGlDouble/N << endl;
        outTxt << "Время жизни одной частицы: " << fullTimeGl/N << endl;
        outTxt.close();
    }

    ofstream statTxt("stat.txt");
    if(statTxt.is_open()) {
        statTxt << "Время работы цикла(всех процессов): " << fullTimeGl <<
endl;

```

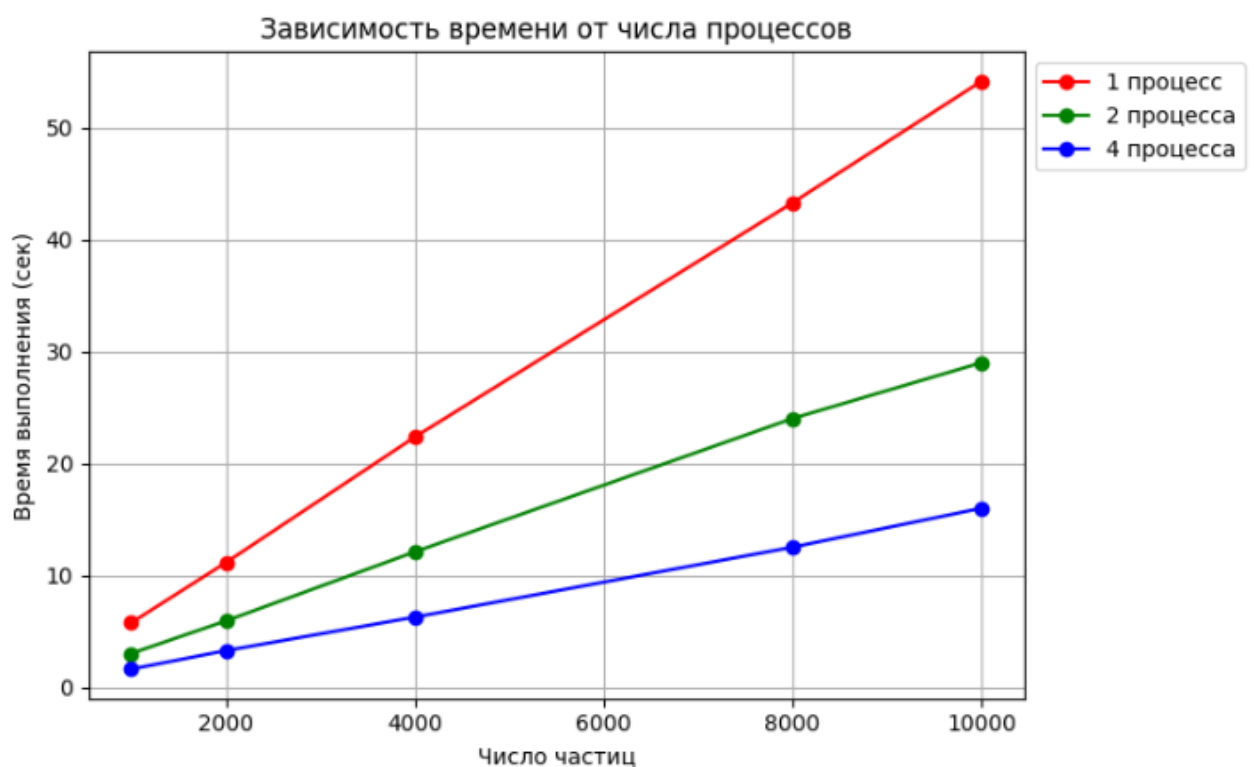
```

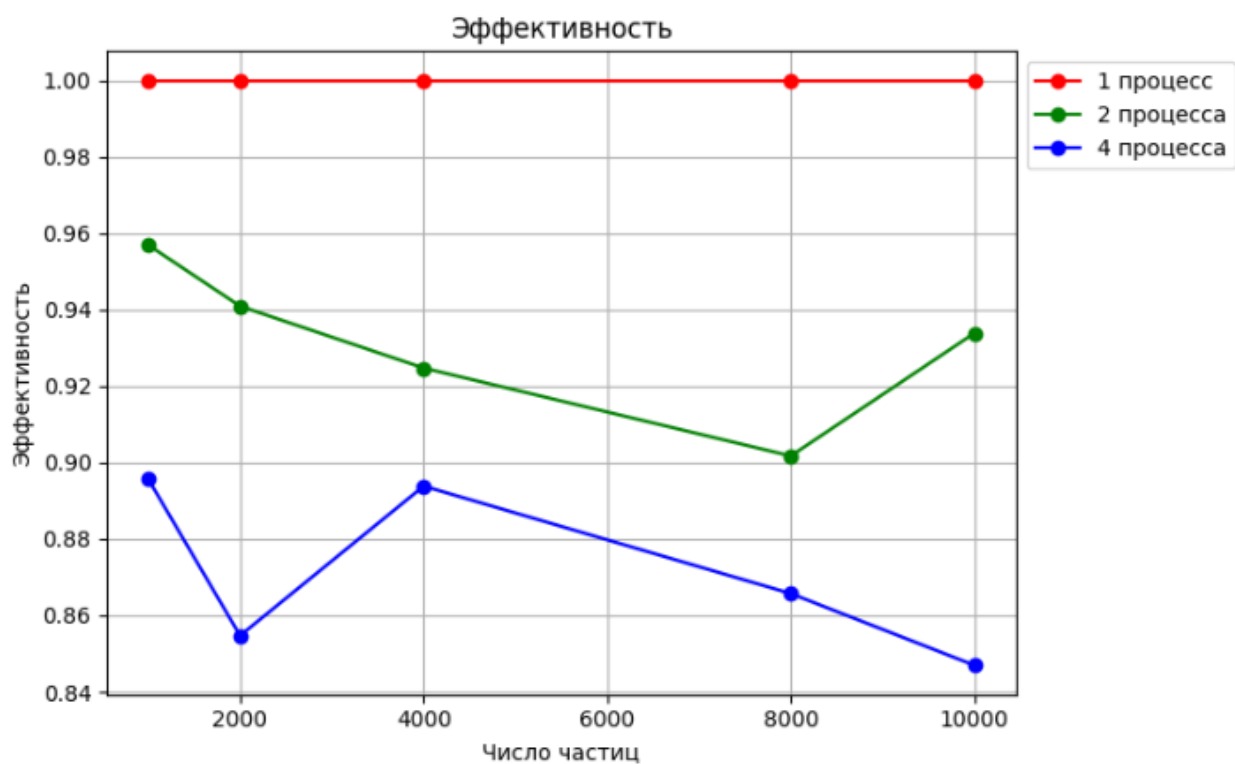
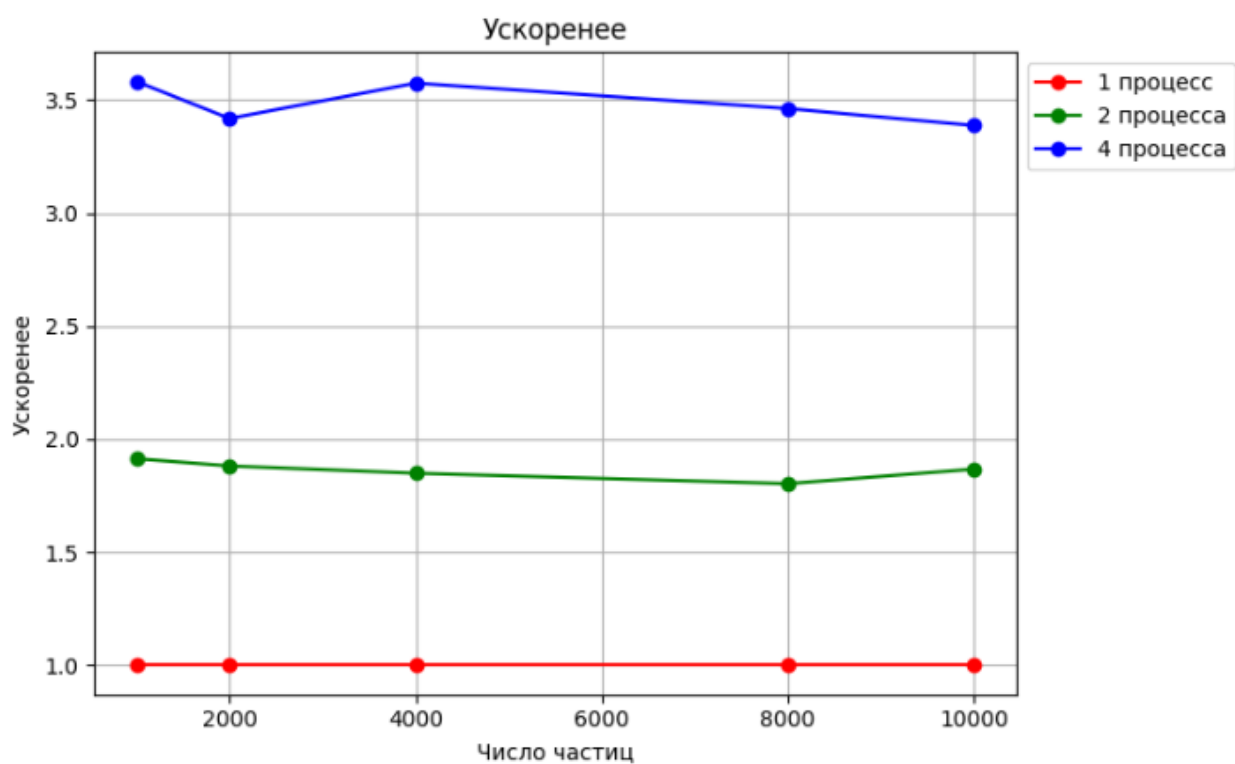
statTxt << "Границы интервала: " << a << " " << b << endl;
statTxt << "Начальная позиция: " << x << endl;
statTxt << "Вероятность перехода вправо: " << p << endl;
statTxt << "Число частиц: " << N << endl;
statTxt << "Число процессов: " << size << endl;
statTxt.close();
}
}
MPI_Finalize();
}

```

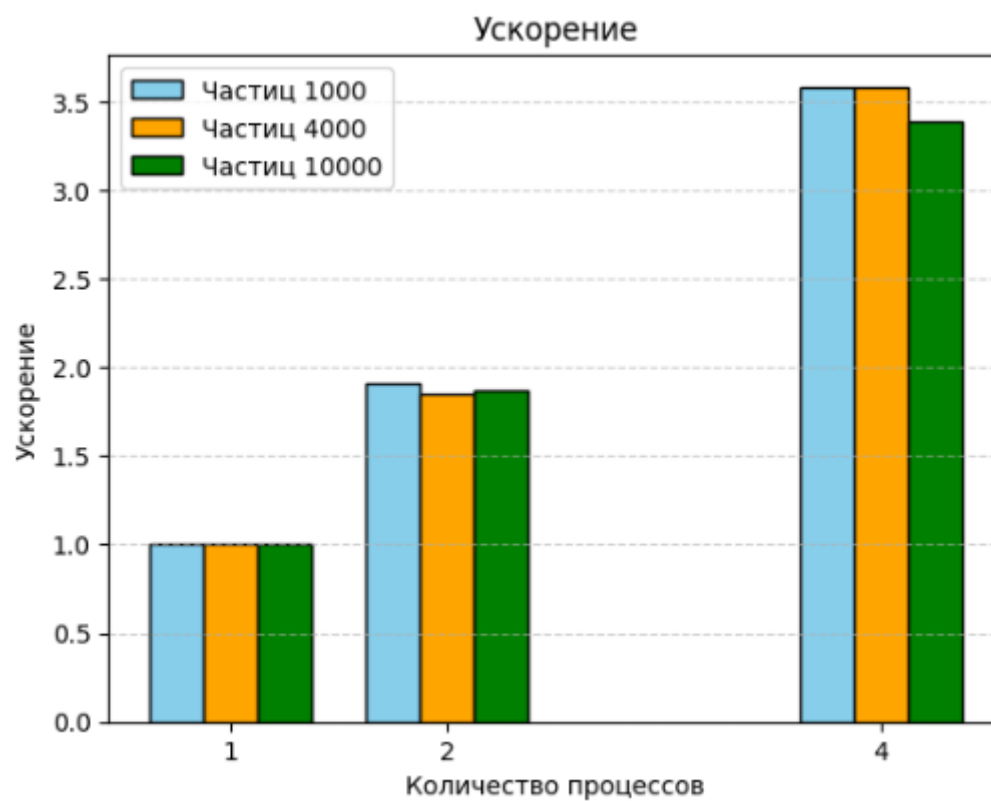
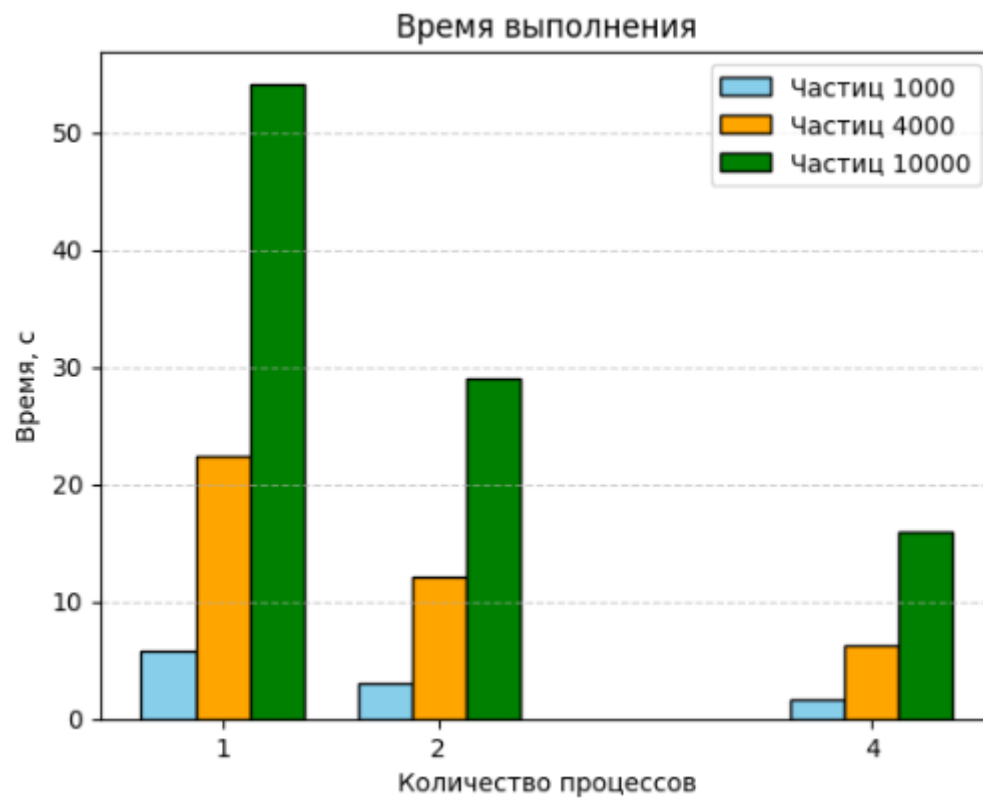
2 Графики

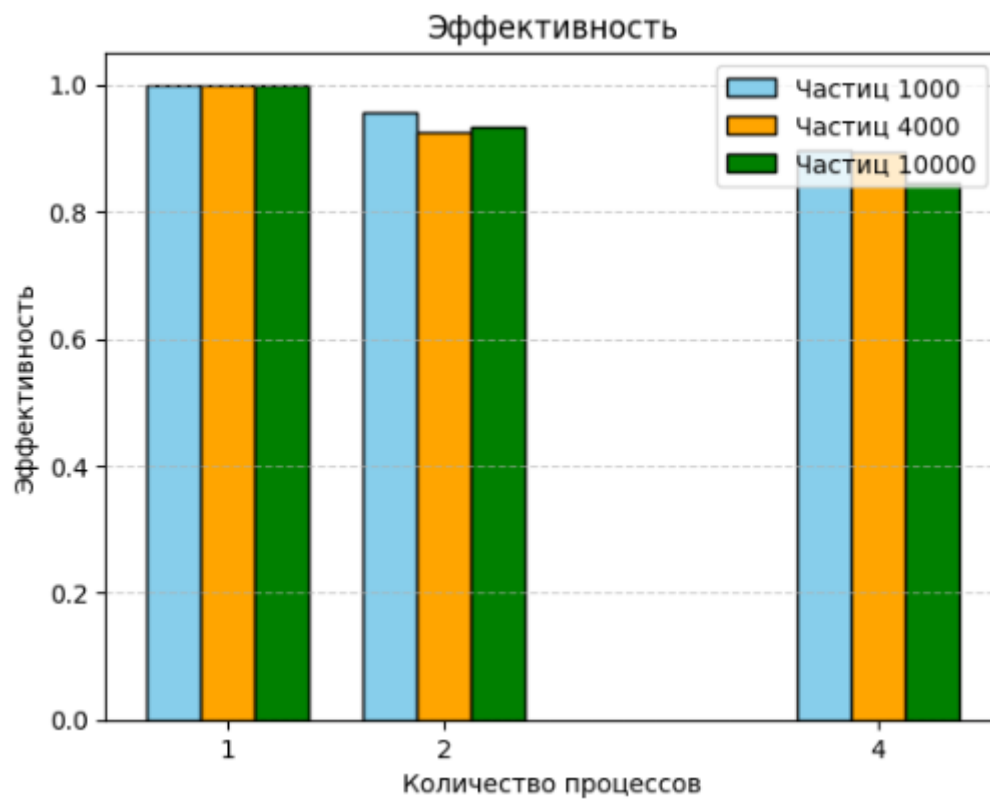
2.1 Фиксированное число процессов





2.2 Фиксированное число частиц





2.3 Число частиц = 10^3 * число процессов

